

PHPMySQL UNIT - 1

Made from the IPU recommended textbook **Programming PHP, First Edition, by Rasmus Lerdorf and Kevin Tatroe (O'Reilly).** and **PHP, MySQL and Apache by Julie C Meloni. Pearson Education**

Unit 1: PHP Fundamentals

1. Introduction to PHP

1.1 Evaluation of PHP (History and Evolution)

- **What is PHP?**
 - PHP stands for **PHP: Hypertext Preprocessor**. It is a widely-used, open-source, server-side scripting language.
 - It is especially suited for web development and can be embedded directly into HTML.
 - Server-side means that the PHP code is executed on the server, and the result (plain HTML) is sent back to the browser. The client cannot see the underlying PHP code.
- **History:**
 - PHP was created by **Rasmus Lerdorf** in 1994.
 - **PHP/FI (Version 1 & 2):** Initially, it was a set of CGI binaries in C called Personal Home Page Tools, used to track visits to his online resume. He released it as "Personal Home Page/Forms Interpreter" or PHP/FI in 1995.
 - **PHP 3 (1998):** Zeev Suraski and Andi Gutmans rewrote the core parser, forming the basis of PHP 3. This version strongly established many of PHP's features like object-oriented support and extensibility.
 - **PHP 4 (2000):** Powered by the new **Zend Engine**, this version brought significant improvements in performance and features.
 - **PHP 5 (2004):** A major update with the Zend Engine II, introducing a much more robust object-oriented model, try/catch exception handling, and improved MySQL support.
 - **PHP 7 (2015):** A revolutionary release with the Zend Engine 3 (PHP-NG), which brought massive performance improvements, often twice as fast as PHP 5.6, along with new features like scalar type declarations and the null coalescing operator.
 - **PHP 8 (2020 onwards):** Continues to add modern language features like the JIT (Just-In-Time) compiler, named arguments, attributes, and union types, further improving performance and code quality.

1.2 Basic Syntax

- **PHP Tags:**

- PHP code must be enclosed in special start and end tags. The parser executes any code within these tags.
- **Standard (Preferred) Tag:** `<?php ... ?>` This is the most portable and recommended way to embed PHP code.
- **Short Tag:** `<? ... ?>` This is a shorthand but must be enabled in the `php.ini` file (`short_open_tag=On`). It is discouraged as it may conflict with XML declarations.
- **Script Tag:** `<script language="php"> ... </script>` This is another way, similar to JavaScript embedding.

- **Statements and Semicolons:**

- A statement is a single instruction in PHP.
- Every PHP statement must end with a semicolon (`;`). This separates one instruction from the next.
- The closing PHP tag `?>` automatically implies a semicolon, so it can be omitted on the very last line of a script.

- **Comments:**

- Comments are parts of the code that are ignored by the parser. They are used to add explanatory notes.
- **Single-line C++ style:** `// This is a comment`
- **Single-line Shell style:** `# This is also a comment`
- **Multi-line C style:** `/* This is a comment that can span multiple lines */`

- **Case Sensitivity:**

- **Case-Insensitive:** Function names, class names, and keywords (like `echo`, `if`, `while`) are not case-sensitive. `ECHO` is the same as `echo`.
- **Case-Sensitive:** Variable names are case-sensitive. `$name`, `$Name`, and `$NAME` are three different variables.

1.3 Defining Variables and Constants

- **Variables:**

- A variable is a container for storing information like strings, numbers, or arrays.
- **Rules for Naming:**
 - All variable names in PHP must start with a dollar sign (`$`).
 - The first character after the `$` must be a letter or an underscore (`_`).
 - Subsequent characters can be letters, numbers, or underscores.
 - They are case-sensitive.

```
<?php
$name = "John Doe"; // A string variable
$age = 25;           // An integer variable
```

```
$is_student = true; // A boolean variable

echo "Name: $name, Age: $age";
?>
```

- **Constants:**

- A constant is an identifier for a simple value that cannot be changed during the execution of the script.
- **Defining Constants:** Use the `define()` function. It takes two arguments: the name of the constant and its value.
- **Rules for Naming:**
 - Constant names do not start with a `$`.
 - By convention, constant names are always in uppercase.
 - They can only store scalar values (boolean, integer, float, and string).

```
<?php
define("SITE_URL", "http://www.example.com");
define("MAX_LOGIN_ATTEMPTS", 3);

echo "Welcome to " . SITE_URL;
?>
```

1.4 PHP Data Types

PHP is a loosely-typed language, meaning you do not need to declare the data type of a variable; PHP automatically converts the variable to the correct data type depending on its value.

- **Scalar Types (for single values):**

- **Integer:** Whole numbers, without a decimal point (e.g., `10`, `-200`, `4195`).
- **Float (or Double):** Numbers with a decimal point or a number in exponential form (e.g., `3.14`, `-0.012`, `4e7`).
- **String:** A sequence of characters enclosed in quotes (e.g., `"Hello World"`). Can be in single (`'`) or double (`"`) quotes.
- **Boolean:** Represents two possible states: `true` or `false`. Used in conditional testing.

- **Compound Types (for multiple values):**

- **Array:** An ordered map that associates values with keys. Used to store multiple values in a single variable.
- **Object:** An instance of a class. It stores data (properties) and information on how to process that data (methods).

- **Special Types:**

- **Resource:** A special variable holding a reference to an external resource, like a database connection or a file handle.
- **NULL:** A special data type that has only one value: `NULL`. A variable is of type `NULL` if it has been assigned the constant `NULL` or has not been set to any value yet.

1.5 Operator and Expression

An expression is anything that has a value. An operator is a symbol that tells the parser to perform a specific mathematical or logical manipulation.

Operator Type	Symbols	Description & Example
Arithmetic	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> (Modulus)	Performs standard mathematical operations. <code>\$c = \$a + \$b;</code>
Assignment	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>.=</code>	Assigns values. <code>\$x = 10;</code> , <code>\$x += 5;</code> (same as <code>\$x = \$x + 5;</code>)
Comparison	<code>==</code> , <code>===</code> , <code>!=</code> , <code><></code> , <code>!==</code> , <code>></code> , <code><</code> , <code>>=</code> , <code><=</code>	Compares two values. Used in conditional statements.
Logical	<code>and</code> (<code>&&</code>), <code>or</code> (<code> </code>)	
Increment/Decrement	<code>++</code> , <code>--</code>	Increases or decreases a variable's value by one. <code>\$counter++;</code>
String	<code>.</code> (Concatenation)	Joins two strings together. <code>\$fullName = \$firstName . " " . \$lastName;</code>

• Difference between `==` and `===`

<code>==</code> (Equal)	<code>===</code> (Identical)
Checks if the values are equal after type juggling.	Checks if the values are equal AND if the data types are the same.
<code>5 == "5"</code> is true .	<code>5 === "5"</code> is false .
Slower, as it involves type conversion.	Faster, as it does a strict comparison.

2. Decisions and Loops

2.1 Making Decisions

- **if Statement:** Executes a block of code if a specified condition is true.

```
$age = 20;
if ($age >= 18) {
    echo "You are eligible to vote.";
}
```

- **if...else Statement:** Executes one block of code if the condition is true, and another block if it is false.

```
$score = 75;
if ($score >= 60) {
    echo "You passed!";
} else {
    echo "You failed.";
}
```

- **if...elseif...else** **Statement:** Executes different blocks of code for more than two conditions.

```
$grade = 'B';
if ($grade == 'A') {
    echo "Excellent!";
} elseif ($grade == 'B') {
    echo "Good job!";
} else {
    echo "You need to work harder.";
}
```

- **switch** **Statement:** Selects one of many blocks of code to be executed based on the value of a variable.
 - The **break** keyword stops the execution of more code in the switch block.
 - The **default** keyword specifies the code to run if there is no case match.

```
$favColor = "red";
switch ($favColor) {
    case "red":
        echo "Your favorite color is red!";
        break;
    case "blue":
        echo "Your favorite color is blue!";
        break;
    default:
        echo "Your favorite color is neither red nor blue!";
}
```

2.2 Doing Repetitive Tasks with Looping

- **while** **Loop:** Executes a block of code as long as the specified condition is true. The condition is checked *before* each iteration.

```
$i = 1;
while ($i <= 5) {
    echo "The number is: $i <br>";
    $i++;
}
```

- **do...while Loop:** Executes a block of code once, and then repeats the loop as long as the specified condition is true. The condition is checked *after* each iteration.

```
$i = 1;
do {
    echo "The number is: $i <br>";
    $i++;
} while ($i <= 5);
```

- **for Loop:** Executes a block of code a specified number of times. It is used when you know in advance how many times the script should run.

- **Syntax:** `for (initializer; condition; incrementer)`

```
for ($i = 0; $i < 10; $i++) {
    echo "The number is $i <br>";
}
```

- **foreach Loop:** Works only on arrays, and is used to loop through each key/value pair in an array.

```
$colors = array("red", "green", "blue");
foreach ($colors as $value) {
    echo "$value <br>";
}
```

2.3 Mixing Decisions and Looping with HTML

PHP's alternate syntax for control structures is useful for embedding logic within HTML without messy curly braces. Replace `{` with `:` and `}` with `end...;`.

```
<ul>
    <?php for ($i = 1; $i <= 5; $i++): ?>
        <li>
            <?php if ($i % 2 == 0): ?>
                <span style="color:blue;">Item <?php echo $i; ?> (Even)
</span>
            <?php else: ?>
                <span style="color:red;">Item <?php echo $i; ?> (Odd)</span>
            <?php endif; ?>
        </li>
    <?php endfor; ?>
</ul>
```

3. Functions

3.1 What is a Function?

- A function is a block of statements that can be used repeatedly in a program.

- It will not execute automatically when a page loads.
- It will be executed by a call to the function.
- **Advantages:**
 - **Reusability:** Write the code once and use it many times.
 - **Readability:** Makes the code easier to read and understand.
 - **Maintainability:** Easier to fix bugs or update logic in one central place.

3.2 Define a Function

- A user-defined function declaration starts with the word `function`, followed by the function name, parentheses `()`, and curly braces `{}`.
- Function names follow the same rules as other labels in PHP but are case-insensitive.
- Parameters (arguments) are specified inside the parentheses.

```
<?php
// Function without parameters
function writeMessage() {
    echo "Hello world!";
}

// Function with parameters
function addNumbers($num1, $num2) {
    $sum = $num1 + $num2;
    echo "Sum of the two numbers is: $sum";
}

// Function that returns a value
function multiplyNumbers($num1, $num2) {
    return $num1 * $num2;
}

// Calling the functions
writeMessage(); // Outputs "Hello world!"
echo "<br>";
addNumbers(5, 10); // Outputs "Sum of the two numbers is: 15"
echo "<br>";
$result = multiplyNumbers(5, 3); // $result becomes 15
echo "The result is: $result";
?>
```

3.3 Call by Value and Call by Reference

This determines how arguments are passed to a function.

Basis	Call by Value (Default)	Call by Reference
Mechanism	A copy of the argument's value is passed to the function.	The memory address (a reference) of the argument is passed to the function.
Effect on Original	Changes made to the parameter inside the function do not affect the original variable outside the function.	Changes made to the parameter inside the function will affect the original variable.
Syntax	Normal function call. <code>myFunction(\$var);</code>	The parameter in the function definition is prefixed with an ampersand (&). function <code>myFunction(&\$var)</code>

- **Example: Call by Value**

```
function addFive($num) {
    $num += 5;
}

$originalNum = 10;
addFive($originalNum);
echo $originalNum; // Outputs 10
```

- **Example: Call by Reference**

```
function addFiveByRef(&$num) {
    $num += 5;
}

$originalNum = 10;
addFiveByRef($originalNum);
echo $originalNum; // Outputs 15
```

3.4 Recursive Function

- A recursive function is a function that calls itself.
- It is useful for tasks that can be broken down into smaller, similar sub-tasks.
- Every recursive function must have a **base case**—a condition that stops the recursion to prevent an infinite loop.
- **Example: Factorial Calculation**

```
function factorial($n) {
    // Base case: if n is 0, factorial is 1
    if ($n == 0) {
        return 1;
    } else {
```



```

        // Recursive step
        return $n * factorial($n - 1);
    }
}

echo "Factorial of 5 is: " . factorial(5); // Outputs 120

```

4. Strings

4.1 Creating and Accessing Strings

- **Single Quoted ('...'):**

- The simplest way to create a string.
- Variables (`$name`) and most escape sequences (`\n`) are **not** interpreted. They are treated as literal text.
- Only `\'` (single quote) and `\\` (backslash) are interpreted.

- **Double Quoted ("..."):**

- Variables are interpreted (variable interpolation).
- Many escape sequences are interpreted, like `\n` (newline), `\t` (tab), `\$` (literal dollar sign).

- **Heredoc:**

- Used for creating large, multi-line strings.
- It starts with `<<<` followed by an identifier, then the string content, and finally the same identifier on a new line to close it. The closing identifier must be at the beginning of the line with no other characters.
- Variables are interpolated just like in double-quoted strings.

```

$name = "PHP";
$text = <<<EOT
This is a multi-line string.
The value of $name is $name.
It's very useful.
EOT;
echo $text;

```

- **Accessing Characters:**

- You can access individual characters in a string using array-like syntax (square brackets `[]`). String indexing is zero-based.

```

$str = "Hello";
echo $str[0]; // Outputs 'H'
echo $str[1]; // Outputs 'e'

```

4.2 Searching & Replacing Strings

Function	Description	Example
<code>strlen(\$str)</code>	Returns the length of a string.	<code>strlen("hello"); // returns 5</code>
<code>strpos(\$haystack, \$needle)</code>	Finds the position of the first occurrence of a substring in a string. Returns <code>false</code> if not found.	<code>strpos("hello world", "world"); // returns 6</code>
<code>strstr(\$haystack, \$needle)</code>	Finds the first occurrence of a substring and returns the rest of the string from that point.	<code>strstr("hello world", "world"); // returns "world"</code>
<code>substr(\$str, \$start, \$len)</code>	Returns a part of a string.	<code>substr("hello world", 6, 5); // returns "world"</code>
<code>str_replace(\$search, \$replace, \$subject)</code>	Replaces all occurrences of the search string with the replacement string.	<code>str_replace("world", "PHP", "hello world"); // returns "hello PHP"</code>

4.3 Formatting Strings

- **Case Conversion:**

- `strtolower($str)`: Converts string to lowercase.
- `strtoupper($str)`: Converts string to uppercase.
- `ucfirst($str)`: Converts the first character of a string to uppercase.
- `ucwords($str)`: Converts the first character of each word in a string to uppercase.

- **Whitespace Trimming:**

- `trim($str)`: Removes whitespace from both the beginning and end of a string.
- `ltrim($str)`: Removes whitespace from the beginning of a string.
- `rtrim($str)`: Removes whitespace from the end of a string.

- **Advanced Formatting with `printf()` and `sprintf()`:**

- `printf()`: Outputs a formatted string to the browser.
- `sprintf()`: Returns the formatted string as a variable, without outputting it.
- They use format specifiers like `%s` (for strings), `%d` (for integers), and `%.2f` (for floats with 2 decimal places).

```
$item = "laptop";
$price = 1200.50;
printf("The %s costs %.2f", $item, $price); // Outputs "The laptop costs $1200.50"
```

4.4 String Related Library Functions

- **explode(\$delimiter, \$string)**: Splits a string into an array by a specified delimiter.

```
$str = "one,two,three";  
$arr = explode(",", $str);  
print_r($arr); // Outputs Array ( [0] => one [1] => two [2] => three )
```

- **implode(\$glue, \$array)**: Joins array elements into a single string with a specified glue.

```
$arr = array("one", "two", "three");  
$str = implode(" ", $arr);  
echo $str; // Outputs "one two three"
```

- **nl2br(\$string)**: Inserts HTML line breaks (`<br>` or `<br />`) before all newlines in a string.
 - **htmlspecialchars(\$string)**: Converts special characters to HTML entities to prevent XSS attacks (e.g., `<` becomes `<`).
-